SVP Engagement Analytics Dashboard

Backend v2

Stack: PostgreSQL · Express.js · Node.js · TypeScript · Prisma ORM

📋 Table of Contents

1. Release Strategy
2. v1.0 Core
    - Database Schema
    - API Endpoints
3. v1.1 Recurring Appointments

🎯 Release Strategy

Version	Scope
v1.0	DB Schema, Auth, Basic CRUD
v1.1	Recurring Appointments
v2.0	Analytics & DataMart
v2.1	Import/Export
v3.0	Partner Portal

> **Principle:** Core (v1.x) is hard to change later. Features (v2.x) are built on top.

🔴 v1.0 CORE

Database Schema

Design Principles

Principle	Implementation	Database Description
UUIDs	PostgreSQL generates all PKs — Express uses `RETURNING` to fetch	`partner_id UUID PRIMARY KEY DEFAULT gen_random_uuid()`
Computed Active	Prisma dynamically computes `is_active` at query time	Filter: `start_date <= TODAY AND (end_date IS NULL OR end_date >= TODAY)`
Multi-tenant	`chapter_id` on every table — filter all queries	`CREATE INDEX idx_partners_chapter ON partners(chapter_id)` + always `WHERE chapter_id = $1`
Timestamps	PostgreSQL auto-manages — Express never touches	`created_at TIMESTAMPTZ DEFAULT NOW()` + trigger auto-updates `modified_at` on UPDATE · All stored in UTC
Email Uniqueness	Unique per chapter — same email allowed across chapters	`CREATE UNIQUE INDEX idx_partners_email ON partners(chapter_id, email)`
Date Validation	`start_date ≤ end_date` enforced at DB and app layer — null end_date means open-ended	`CHECK (start_date <= COALESCE(end_date, '9999-12-31'))`
Standardized Types	`group_type` and `appointment_type` use lookup tables — prevents free-text pollution	FK → `group_types` / `appointment_types` tables

Field Length Standards

Field Type	Length	Examples
Names (user)	`VARCHAR(150)`	Admin names: "Priya Sharma"
Names (partner)	`VARCHAR(150)`	Long Indian names with titles
Names (investee)	`VARCHAR(200)`	Descriptive NGO names
Names (group)	`VARCHAR(150)`	"Mentorship Cohort 2026"
Names (chapter)	`VARCHAR(100)`	"SVP India - Bangalore"
Email	`VARCHAR(255)`	RFC 5321 standard
URLs	`VARCHAR(500)`	LinkedIn with tracking params
Types/Categories	`VARCHAR(150)`	"Mentorship", "Board"

Field Type	Length	Examples
Password Hash	`VARCHAR(72)`	bcrypt (60 chars + buffer)

> 💡 PostgreSQL VARCHAR only stores actual characters. Length is validation, not storage.

Tables

`chapters`

Column	Type	Constraints
chapter_id	UUID	PK
chapter_name	VARCHAR(100)	NOT NULL
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

`users`

Column	Type	Constraints
user_id	UUID	PK
chapter_id	UUID	FK → chapters
user_type	ENUM	'ADMIN', 'PARTNER'
name	VARCHAR(150)	NOT NULL
email	VARCHAR(255)	UNIQUE per chapter (login + communication)
password_hash	VARCHAR(72)	bcrypt
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

> **v3.0 Partner Login:** Partner user created with same email as `partners.email`. System matches by email + chapter_id (no FK needed, follows Open/Closed principle).

`partners`

Column	Type	Constraints

Column	Type	Constraints
partner_id	UUID	PK
chapter_id	UUID	FK → chapters
partner_name	VARCHAR(150)	NOT NULL
email	VARCHAR(255)	UNIQUE per chapter (login + communication)
linkedin_url	VARCHAR(500)	Optional
primary_partner_id	UUID	FK → partners, NULL = is primary
start_date	DATE	NOT NULL
end_date	DATE	Optional
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

> **Primary vs Secondary Partners:**
>
> - `primary_partner_id = NULL` → This is a **primary partner** (main person)
> - `primary_partner_id = <id>` → This is a **secondary partner** (family member of primary)

> **Delete Policy:** Cannot delete if referenced in `group_partners` or `appointment_partners` or `recurring_appointment_partners`. Block with error message.

`investees`

Column	Type	Constraints
investee_id	UUID	PK
chapter_id	UUID	FK → chapters
investee_name	VARCHAR(200)	NOT NULL
email	VARCHAR(255)	NOT NULL (communication only, not unique)
start_date	DATE	NOT NULL
end_date	DATE	Optional
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

> **Delete Policy:** Cannot delete if referenced in `groups` or `appointments` or `recurring_appointments`. Block with error message.

`group_types`

Column	Type	Constraints
group_type_id	UUID	PK, DEFAULT gen_random_uuid()
chapter_id	UUID	FK → chapters
type_name	VARCHAR(150)	NOT NULL
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

> **Unique Constraint:** `UNIQUE(chapter_id, type_name)` to prevent duplicate types within a chapter.
> **Delete Policy:** Cannot delete if referenced by any `groups` or `appointments`. Block with error message.

`appointment_types`

Column	Type	Constraints
appointment_type_id	UUID	PK, DEFAULT gen_random_uuid()
chapter_id	UUID	FK → chapters
type_name	VARCHAR(150)	NOT NULL
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

> **Unique Constraint:** `UNIQUE(chapter_id, type_name)` to prevent duplicate types within a chapter.
> **Delete Policy:** Cannot delete if referenced by any `appointments` or `recurring_appointments`. Block with error message.

`groups`

Column	Type	Constraints

Column	Type	Constraints
group_id	UUID	PK
chapter_id	UUID	FK → chapters
group_name	VARCHAR(150)	NOT NULL
group_type_id	UUID	FK → group_types (optional)
investee_id	UUID	FK → investees (optional)
start_date	DATE	NOT NULL
end_date	DATE	Optional
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

> **Groups as UI Helpers:** Groups organize partners for easier selection. When creating an appointment via a group, the frontend pre-fills the partner list from the group's active `group_partners` and pre-fills the appointment type from the group's `group_type_id` — the admin can review and modify both before submitting. The backend receives a plain list of `partner_ids` and an `appointment_type_id`; it has no concept of "this came from a group".

> **Delete Policy:** Cannot delete if referenced by `recurring_appointments`. Block with error message. **Cascade:** If group is deleted, all `group_partners` entries for this group are automatically deleted (ON DELETE CASCADE).

`group_partners`

Column	Type	Constraints
group_partner_id	UUID	PK
chapter_id	UUID	FK → chapters
group_id	UUID	FK → groups
partner_id	UUID	FK → partners
start_date	DATE	NOT NULL
end_date	DATE	Optional
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

> **Date Range Policy:**
> `group_partner` dates must fall within the **intersection** of `partner` dates AND `group` dates.
> **Default:** Auto-set to intersection when adding partner to group.
> **Manual Override:** Admin can narrow the range further (e.g., partner on leave, temporary role).
> **Null End Dates:** A `null` end date on any of the three records (partner, group, or group_partner) means "no end / open-ended" — treated as `9999-12-31` in all comparisons.
> **Validation:** Enforced at application layer only.
> **Logic:** `MAX(partner.start_date, group.start_date) ≤ group_partner.start_date ≤ COALESCE(group_partner.end_date, '9999-12-31') ≤ MIN(COALESCE(partner.end_date, '9999-12-31'), COALESCE(group.end_date, '9999-12-31'))`

`appointments`

Column	Type	Constraints
appointment_id	UUID	PK
chapter_id	UUID	FK → chapters
rec_appointment_id	UUID	FK → recurring_appointments (optional)
appointment_type_id	UUID	FK → appointment_types (optional)
group_type_id	UUID	FK → group_types (optional)
occurrence_date	DATE	Set to `start_at::DATE` (in IST) for all appointments
start_at	TIMESTAMPTZ	NOT NULL
end_at	TIMESTAMPTZ	NOT NULL
duration_minutes	INTEGER	GENERATED (computed as `EXTRACT(EPOCH FROM (end_at - start_at)) / 60`)
investee_id	UUID	FK → investees (optional)
status	ENUM	'PENDING', 'COMPLETED', 'CANCELLED'
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

> **Time Policy:** `start_at <= end_at` enforced at database level and validated at application layer.

> **Groups are Frontend-Only:** The backend has no concept of "created from a group". When using a group, the frontend pre-fills the active group_partner list and type field `group_type_id` from the group's `group_type_id` — the admin reviews and edits both — and then submits a plain list of `partner_ids`, and `group_type_id`. No `group_id` is stored on the appointment.

> **Recurring Reference:** `rec_appointment_id` is only set when system materializes a recurring appointment. If recurring appointment template is deleted, system sets `rec_appointment_id = NULL` in all materialized appointments (ON DELETE SET NULL).

> **Idempotency:** `CREATE UNIQUE INDEX idx_unique_occurrence ON appointments (rec_appointment_id, occurrence_date) WHERE rec_appointment_id IS NOT NULL` — prevents duplicate materializations. Standalone appointments (where `rec_appointment_id IS NULL`) are excluded.

> **Delete Policy:** Can delete appointment directly (will cascade delete `appointment_partners`).

> **Partner Add:** When adding partners to an appointment (at creation or update), each partner must have `is_active = true` on `start_at::DATE` — i.e., `partner.start_date ≤ DATE(start_at)`.

`appointment_partners`

Column	Type	Constraints
app_partner_id	UUID	PK
chapter_id	UUID	FK → chapters
appointment_id	UUID	FK → appointments, ON DELETE CASCADE
partner_id	UUID	FK → partners
is_present	BOOLEAN	DEFAULT false
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

> **Unique Constraint:** `UNIQUE(appointment_id, partner_id)` to prevent duplicate partner entries in same appointment.

API Endpoints

Base URL: `/api/v1`

Auth Header: `Authorization: Bearer <jwt>`

Error Handling: Consistent JSON error format for API failures (e.g., standard `400 Bad Request` for constraint violations/validation errors, and `404 Not Found` for missing resources).

> **Note:** Backend handles filtering and pagination. Frontend handles search and sorting of results. For frontend search/dropdowns, the frontend should request `?limit=all` (or similar, like `?paginate=false`) to retrieve the full, unpaginated list of records from the backend.

Auth

Method	Endpoint	Action
POST	`/auth/login`	Login with email/password → returns JWT token
POST	`/auth/logout`	Invalidate current JWT token
GET	`/auth/me`	Get current logged-in user details

Partners

Method	Endpoint	Action
GET	`/partners`	**List:** Pagination (`page`, `limit`) · **Filter:** `is_active` (true/false/all), `primary_partner_id` (null/not null/all) · Returns all matching partners (search/sort on frontend)
GET	`/partners/:id`	Get single partner profile
POST	`/partners`	Create new partner
PUT	`/partners/:id`	Update partner details
DELETE	`/partners/:id`	Delete partner — blocked if referenced

Investees

Method	Endpoint	Action
GET	`/investees`	**List:** Pagination (`page` , `limit`) · **Filter:** `is_active` (true/false/all) · Returns all matching investees (search/sort on frontend)
GET	`/investees/:id`	Get single investee profile
POST	`/investees`	Create new investee organization
PUT	`/investees/:id`	Update investee details
DELETE	`/investees/:id`	Delete investee — blocked if referenced

Groups

Method	Endpoint	Action
GET	`/groups`	**List:** Pagination (`page` , `limit`) · **Filter:** `is_active` (true/false/all) · Returns all matching groups (search/sort on frontend)
GET	`/groups/:id`	Get group details with group partners and investee
POST	`/groups`	Create new group with initial partners array and other metadata (name, type, start_date, end_date, investee)
PUT	`/groups/:id`	Update group metadata (name, type, start_date, end_date, investee)
DELETE	`/groups/:id`	Delete group — blocked if referenced in recurring appointments
PUT	`/groups/:id/partners`	**Replace partner list:** Accept the full array of `{ partner_id, start_date, end_date }` — replaces the group's entire partner list (adds new, updates existing, removes omitted)

System Lookup Types

Method	Endpoint	Action
GET	`/group-types`	List all group types for the chapter
POST	`/group-types`	Create new group type — if a type with the same name already exists in this chapter, returns the existing one instead of creating a duplicate
DELETE	`/group-types/:id`	Delete group type — blocked if referenced by any groups or appointments

Method	Endpoint	Action
GET	`/appointment-types`	List all appointment types for the chapter
POST	`/appointment-types`	Create new appointment type — if a type with the same name already exists in this chapter, returns the existing one instead of creating a duplicate
DELETE	`/appointment-types/:id`	Delete appointment type — blocked if referenced by any appointments or recurring appointments

Appointments

Method	Endpoint	Action
GET	`/appointments`	**List:** Pagination (`page` , `limit`) · **Filter:** `appointment_date` (from/to range), `status` (PENDING/COMPLETED/CANCELLED) · Returns all matching appointments (search/sort on frontend)
GET	`/appointments/:id`	Get appointment with partners and attendance, investee, rec_appointment
POST	`/appointments`	Create PENDING appointment (standalone or from group)
PUT	`/appointments/:id`	Update appointment (start_at, end_at, appointment_type_id, group_type_id, investee, array of partners, status)
PATCH	`/appointments/:id/complete`	Mark complete and update attendance
DELETE	`/appointments/:id`	Delete appointment (cascades to `appointment_partners`)

> **Note:** The `/complete` endpoint updates status to COMPLETED AND attendance in one requst.

🟠 v1.1 RECURRING APPOINTMENTS

> **Scope:** Adds 2 tables + background job — materializes templates as real appointments

---

Database Schema

Design Philosophy

Concept	Implementation
Time Materialization	`start_at = (occurrence_date + start_time) AT TIME ZONE 'Asia/Kolkata'`, `end_at = start_at + duration_minutes` — all times stored as UTC
Background Materialization	Cron job runs nightly at 2 AM, materializes all unmaterialized occurrences for yesterday as PENDING appointments
Template Immutability	Template changes only affect the unmaterialized occurrences
Participating Partners	Union of all active `group_partners` on the occurenece date + any partners explicitly listed in `recurring_appointment_partners`. Deduplicated before creating `appointment_partners`.

Tables

`recurring_appointments`

Column	Type	Constraints
rec_appointment_id	UUID	PK
chapter_id	UUID	FK → chapters
group_id	UUID	FK → groups (optional)
appointment_type_id	UUID	FK → appointment_types (optional)
start_time	TIME	NOT NULL
duration_minutes	INTEGER	NOT NULL
rrule	TEXT	iCalendar RRule format, NOT NULL
investee_id	UUID	FK → investees (optional)
start_date	DATE	NOT NULL
end_date	DATE	NOT NULL
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

RRule Examples:

```
FREQ=WEEKLY;BYDAY=MO,WE,FR                // Every Mon, Wed, Fri
FREQ=MONTHLY;BYMONTHDAY=1,15              // 1st and 15th of month
FREQ=WEEKLY;INTERVAL=2;BYDAY=TU           // Every 2 weeks on Tuesday
```

> **Delete Policy:** Delete sets `rec_appointment_id = NULL` in materialized appointments (ON DELETE SET NULL), and cascade delete recurring_appointment_partners (ON DELETE CASCADE).

> **Application Validations:**
>
> - `rrule` must be a valid iCalendar RRule string — validated at application layer on create and update
> - `end_date` can be set to at most 1 year from the date of the request (create or update)

`recurring_appointment_partners`

Column	Type	Constraints
rec_app_partner_id	UUID	PK
chapter_id	UUID	FK → chapters
rec_appointment_id	UUID	FK → recurring_appointments, ON DELETE CASCADE
partner_id	UUID	FK → partners
created_at	TIMESTAMPTZ	DEFAULT NOW()
modified_at	TIMESTAMPTZ	DEFAULT NOW()

> **Unique Constraint:** `UNIQUE(rec_appointment_id, partner_id)` to prevent duplicate partner entries in same recurring appointment.

API Endpoints

Base URL: `/api/v1`

Recurring Appointments

Method	Endpoint	Action
GET	`/recurring-appointments`	**List:** Pagination (`page` , `limit`) · **Filter:** `is_active` (true/false/all), · Returns all matching templates (search/sort on frontend)

Method	Endpoint	Action
GET	`/recurring-appointments/:id`	Get template with recurring appointment partners, group, investee
POST	`/recurring-appointments`	Create template with optional partner subset
POST	`/recurring-appointments/:id/materialize`	**Manual Materialization:** Admin materializes specific occurrence `{ occurrence_date: "2024-01-15" }` → creates PENDING appointment
PUT	`/recurring-appointments/:id`	Update template (only affects unmaterialized occurrences), can update start_date, end_date, start_time, duration, appointment_type, investee, group, rrule, array of partners
DELETE	`/recurring-appointments/:id`	Delete — sets `rec_appointment_id = NULL` in appointments table

Background Job (Internal)

Method	Endpoint	Action
POST	`/jobs/materialize-recurring`	Cron job — runs nightly at 2 AM, materializes all unmaterialized occurrences for yesterday as PENDING appointments

---

Materialization Flow

Core Materialization Logic (per `occurrence_date`)

> Shared by both manual and automatic materialization.

1. Check if already materialized for `occurrence_date` — skip if exists (unique index on `rec_appointment_id + occurrence_date`)
2. Collect partners: union of `recurring_appointment_partners` + all active `group_partners` on `occurrence_date`
3. Filter: retain only partners whose active date range includes `occurrence_date`
4. Create appointment: `status = PENDING`, `occurrence_date = occurrence_date`, `start_at = (occurrence_date + template.start_time) AT TIME ZONE 'Asia/Kolkata'`, `end_at = start_at + duration_minutes`, `rec_appointment_id = template.id`

5. Create `appointment_partners` for each partner

Manual Materialization (Admin Action)

Admins can manually materialize any occurrence that has not yet been materialized — past, today, or future.

- **Endpoint:** `POST /recurring-appointments/:id/materialize`
- **Body:** `{ "occurrence_date": "2024-01-15" }`
- **Process:** Runs the core materialization logic above for the given `occurrence_date`
- **After Materialization:** Admin can mark it complete via `PATCH /appointments/:id/complete`

> **Key Principle:** Materialization creates a PENDING appointment → Admin completes it → Status becomes COMPLETED

Automatic Materialization (Cron Job)

Runs nightly at 2 AM. Targets yesterday's date only.

1. Get all active templates where `start_date ≤ yesterday ≤ end_date`
2. For each template, check if yesterday is a valid occurrence according to the template's `rrule` — skip if not a scheduled occurrence
3. Run the core materialization logic for `occurrence_date = yesterday`
4. Commit all inserts as a single transaction